

Tema 4.1. Composición

1. En C, podemos crear estructuras mayores **componiendo** unas con otras, que suelen describirse como "A tiene-un/tiene-varios B". Pon un ejemplo, empleando `struct`, de una línea de puntos, donde puntos tienen dos coordenadas (`x` e `y`), y la línea esta hecha de dos puntos. Incluye una función para calcular la distancia entre puntos y otra para hallar la longitud de una línea.

Ejemplo en **C** que muestra cómo componer estructuras: una **línea** que "tiene dos" **puntos**. Incluye:

- `struct Punto` con coordenadas `x` e `y`.
- `struct Linea` formada por dos `Punto`: `inicio` y `fin`.
- Una función `distanciaPuntos` para calcular la distancia entre dos puntos.
- Una función `longitudLinea` que usa la anterior para calcular la longitud de una línea.

```
#include <stdio.h>
#include <math.h>    // sqrt, pow

// Estructura para un punto en 2D
typedef struct {
    double x;
    double y;
} Punto;

// Estructura para una línea definida por dos puntos
typedef struct {
    Punto inicio;
    Punto fin;
} Linea;

// Distancia euclidiana entre dos puntos
double distanciaPuntos(Punto a, Punto b) {
    double dx = b.x - a.x;
    double dy = b.y - a.y;
    return sqrt(dx*dx + dy*dy);
    // Alternativa equivalente: return hypot(dx, dy); // si se prefiere
}

// Longitud de una línea (distancia entre sus puntos extremos)
double longitudLinea(Linea l) {
    return distanciaPuntos(l.inicio, l.fin);
}

int main(void) {
    Punto p1 = {0.0, 0.0};
    Punto p2 = {3.0, 4.0};    // Distancia 5 (triángulo 3-4-5)

    Linea l = { p1, p2 };
}
```

```
printf("Distancia entre p1 y p2: %.2f\n", distanciaPuntos(p1, p2));
printf("Longitud de la línea: %.2f\n", longitudLinea(l));

return 0;
}
```

Notas y explicación

- **Composición (tiene-un/tiene-varios):** **Línea** tiene dos **Punto** (**inicio** y **fin**). Es una relación de composición típica en C usando **struct**.
- **Cálculo de distancia:** usamos la **distancia euclidiana** en 2D: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
- **math.h:** recuerda enlazar la librería matemática al compilar:

```
gcc main.c -o programa -lm
```

- **hypot:** si tu compilador/entorno lo permite, **hypot(dx, dy)** maneja mejor casos de desbordamiento/subdesbordamiento numérico, aunque para ejercicios básicos **sqrt(dx*dx + dy*dy)** es suficiente.

2. Ahora transforma ese ejemplo a orientación a objetos con Java, para tener un primer ejemplo de **composición** en orientación a objetos. Crea una clase **Punto**, y una clase **Línea**. La clase **Punto** debe tener un método para calcular distancia a otro **Punto** y **Línea** debe tener un método para calcular su longitud. Gracias a la ocultación de información, supera a C, garantizando que los puntos sean inmutables, al igual que la línea, que una vez creada, no queremos que se modifique de qué a qué puntos va dicha línea.

Pasemos el ejemplo a **orientación a objetos en Java** usando **composición** y **ocultación de información**. Haremos que tanto los **puntos** como la **línea** sean **inmutables** (una vez creados, no cambian).

Objetivo

- **Punto:** clase inmutable con coordenadas **x** e **y**, y un método **distanciaA(Punto)**.
- **Línea:** clase inmutable compuesta por **dos Punto** (**inicio** y **fin**), con método **longitud()** que delega en **distanciaA**.

Implementación

Clase **Punto** (inmutable)

```
public final class Punto {
    private final double x;
    private final double y;
```

```

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    /**
     * Distancia euclidiana entre este punto y otro.
     */
    public double distanciaA(Punto otro) {
        if (otro == null) {
            throw new IllegalArgumentException("El punto de destino no puede ser
null");
        }
        double dx = otro.x - this.x;
        double dy = otro.y - this.y;
        return Math.hypot(dx, dy); // estable y legible
    }

    public double getX() { return x; }
    public double getY() { return y; }

    @Override
    public String toString() {
        return "Punto(" + x + ", " + y + ")";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Punto)) return false;
        Punto p = (Punto) o;
        // Comparación directa de double: para geometría básica suele ser
suficiente.
        // Para usos numéricos delicados, considerar una tolerancia (epsilon).
        return Double.compare(p.x, x) == 0 && Double.compare(p.y, y) == 0;
    }

    @Override
    public int hashCode() {
        return java.util.Objects.hash(x, y);
    }
}

```

Clase **Linea** (inmutable, compuesta por dos **Punto**)

```

public final class Linea {
    private final Punto inicio;
    private final Punto fin;

    public Linea(Punto inicio, Punto fin) {
        if (inicio == null || fin == null) {

```

```

        throw new IllegalArgumentException("Los puntos de la línea no pueden
ser null");
    }
    this.inicio = inicio;
    this.fin = fin;
}

/**
 * Longitud de la línea (distancia entre sus extremos).
 */
public double longitud() {
    return inicio.distanciaA(fin);
}

public Punto getInicio() { return inicio; }
public Punto getFin() { return fin; }

@Override
public String toString() {
    return "Linea{" + "inicio=" + inicio + ", fin=" + fin + '}';
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Linea)) return false;
    Linea l = (Linea) o;
    return java.util.Objects.equals(inicio, l.inicio)
        && java.util.Objects.equals(fin, l.fin);
}

@Override
public int hashCode() {
    return java.util.Objects.hash(inicio, fin);
}
}

```

Ejemplo de uso (pequeño main)

```

public class Demo {
    public static void main(String[] args) {
        Punto p1 = new Punto(0.0, 0.0);
        Punto p2 = new Punto(3.0, 4.0); // distancia 5 (triángulo 3-4-5)

        Linea l = new Linea(p1, p2);

        System.out.println("Distancia p1 -> p2: " + p1.distanciaA(p2));
        System.out.println("Longitud de la línea: " + l.longitud());
        System.out.println(l);
    }
}

```

Por qué esto es “composición” e “inmutabilidad”

- **Composición (tiene-dos):** **Línea** **tiene** dos objetos **Punto** (**inicio** y **fin**). No hay herencia; es una relación “parte-de”.
- **Ocultación de información:** los campos son **private** y solo se accede mediante métodos públicos controlados. No hay forma de modificar el estado desde fuera.
- **Inmutabilidad:**
 - Las clases son **final** (no se pueden extender).
 - Los campos son **private final**.
 - No hay **setters**.
 - Los tipos contenidos (**Punto**) también son inmutables.
 - **getInicio()** y **getFin()** devuelven referencias a objetos inmutables, así que no hace falta copia defensiva.

Ventajas: objetos más seguros y simples de razonar, especialmente en entornos concurrentes; además, son naturalmente “thread-safe” (si sus contenidos lo son).

Variantes útiles (opcionales)

- **Fábricas estáticas** si prefieres más legibilidad:

```
public static Linea de(double x1, double y1, double x2, double y2) {  
    return new Linea(new Punto(x1, y1), new Punto(x2, y2));  
}
```

- **Validación adicional:** rechazar líneas degeneradas (**inicio == fin**) si el dominio lo exige.
- **record** (Java 16+): también garantiza inmutabilidad de campos, pero aquí pediste clases; si te interesa, puedo mostrar la versión con **record**.

3. ¿Qué significa la **multiplicidad** en la composición? En el ejemplo anterior, ¿cuál es la multiplicidad entre **Línea** y **Punto**? Indícalo expresando la multiplicidad en ambas direcciones, de **Línea** a **Punto** y de **Punto** a **Línea**.

La **multiplicidad** en composición (y en UML en general) indica **cuántas instancias** de una clase pueden estar relacionadas con una instancia de otra clase. En composición, además, implica **pertenencia fuerte**: los objetos “parte” no pueden existir sin el objeto “todo”.

✓ Multiplicidad en el ejemplo **Línea** – **Punto**

En nuestra implementación, una **Línea** está compuesta exactamente por **dos** puntos: **inicio** y **fin**.

◇ De Línea → Punto

Una **Línea** **tiene exactamente 2 Puntos**.

- Multiplicidad desde **Línea** hacia **Punto**:
2

Es decir:

Cada **Línea** está compuesta por **dos Punto**.

◇ De Punto → Línea

Un **Punto** **puede formar parte de ninguna o de muchas Líneas**.

Por ejemplo, un **Punto(0,0)** podría ser el inicio de varias líneas, o no formar parte de ninguna.

- Multiplicidad desde **Punto** hacia **Línea**:
0..* (cero o muchas)

✓ Resumen

Relación	Multiplicidad
Línea → Punto	2
Punto → Línea	0..*

Esto refleja una composición **fuerte**:

- La línea **contiene** dos puntos.
- Los puntos usados pueden existir independientemente, pero la composición semántica marca que **una Línea no tiene sentido sin sus dos Puntos**.

4. ¿Qué significa composición **fuerte** y composición **débil**? ¿Qué consecuencia implica en relación al ciclo de vida de los objetos? Indica a cuál solemos referirnos como "**asociación o agregación**" y a cuál como "**composición**" propiamente.

🌀 Composición fuerte vs. composición débil

En UML y en el diseño orientado a objetos se distinguen dos relaciones de "todo–parte":

🌀 1. Composición débil

También llamada **agregación** o **asociación** (dependiendo del grado de dependencia).

✓ Qué es

Es una relación “todo–parte” **suave**, en la que el objeto parte **puede existir independientemente** del objeto todo.

- El “todo” **no es dueño** de las partes.
- Si el “todo” muere, las partes **siguen existiendo**.
- La relación es más parecida a “*A tiene un/varios B, pero B no depende de A para existir*”.

✓ Ejemplo típico

- Un **Equipo** tiene **Jugadores**.
Los jugadores siguen existiendo aunque el equipo desaparezca.

✓ Consecuencias en el ciclo de vida

- **No hay dependencia fuerte** entre los ciclos de vida.
- Los objetos “parte” pueden ser creados antes, compartidos y sobrevivir al “todo”.

✓ En UML

- Se representa con un **rombo blanco**: ◇
- Nombre: **Agregación**.

2. Composición fuerte

Lo que normalmente llamamos **composición** en sentido estricto.

✓ Qué es

Es una relación “todo–parte” **fuerte**, donde el objeto parte **no tiene sentido** ni puede existir sin el objeto todo.

- El “todo” **es dueño** de las partes.
- Si el “todo” desaparece, las partes **también deben desaparecer**.
- Es una relación de pertenencia tipo “*forma parte de*” esencial.

✓ Ejemplo típico

- Una **Casa** tiene **Habitaciones**.
Si destruyes la casa, las habitaciones dejan de existir.

✓ Consecuencias en el ciclo de vida

- El ciclo de vida de la parte está **totalmente ligado** al ciclo de vida del todo.
- La parte normalmente se crea **en el constructor** del todo o se le pasa como parte de la creación.
- La parte **no se comparte** con otros objetos.

✓ En UML

- Se representa con un **rombo negro**: ◆
- Nombre: **Composición** (la de verdad).

Resumen comparativo

Tipo de relación	Nombre habitual	Existencia independiente	Ciclo de vida	Compartición
Composición débil	Agregación / Asociación	Sí, las partes existen solas	Independiente	Sí
Composición fuerte	Composición	No, partes dependen del todo	Ligados	No

Aplicándolo al ejemplo **Línea — Punto**

En el ejemplo de antes:

- Una **Línea** está formada *conceptualmente* por **dos Puntos** → esto es una **composición fuerte**: sin línea no hay endpoints con ese rol.
- Pero en la implementación (Java) permitimos que **Puntos existan fuera** de la línea → las instancias no dependen del ciclo de vida de la línea.

Conceptualmente UML: composición fuerte

En código Java (por inmutabilidad y diseño práctico): se comporta más como agregación *pero de cardinalidad fija*.

5. Cuando una clase usa a otra al recibirla o devolverla como parámetro en algún método, al hacer **new** dentro de un método, o al usarlas como variables locales, ¿hablamos de composición o de "**dependencia**"?

Hablamos de **dependencia**, no de composición.

✓ ¿Qué ocurre cuando una clase usa otra *solo* como parámetro, valor de retorno, variable local o con un **new** dentro de un método?

Eso **no** es composición, sino una relación mucho más débil llamada:

Dependencia (también llamada "uso")

Una clase **depende** de otra cuando:

- la **recibe como parámetro** en un método,
- la **devuelve** como valor de retorno,
- la **crea dentro de un método** (**new** local),
- la usa como **variable local**,
- o la menciona **transitoriamente** en algún comportamiento.

En todos esos casos:

- No forma parte del estado permanente del objeto.
- No afecta al ciclo de vida del objeto "parte".
- La relación dura **solo durante la ejecución del método**.

¿Por qué NO es composición?

La **composición** implica que:

- la clase tiene un **atributo** (campo) que es la otra clase,
- y ese atributo es parte de su **estado permanente**,
- y normalmente el ciclo de vida de la parte depende del "todo".

Ejemplo:

Una **Línea** tiene **dos Punto** → *composición*.

Pero si **Línea** recibe un **Punto** como parámetro temporal en un método, eso **no** crea composición.

Resumen claro

Situación	Tipo de relación
A tiene un atributo B	Composición (o agregación según el caso)
A crea B dentro de un método	Dependencia
A recibe B como parámetro	Dependencia
A devuelve B	Dependencia
A usa B como variable local	Dependencia

Conclusión

Cuando una clase **usa** a otra **solo** dentro de métodos (parámetros, retornos, variables locales, **new** local...), hablamos siempre de:

👉 Dependencia, no composición.

6. En el ejemplo anterior de línea y punto, programa la relación entre **Línea** y **Punto** de dos formas. Una **como composición fuerte**, donde el ciclo de vida de los puntos está ligado al de Línea y otra **como composición débil**, donde no.

¡Vamos a codificar las **dos variantes** (composición fuerte y composición débil) del ejemplo **Línea–Punto** en **Java**, manteniendo **inmutabilidad** y explicando cómo cambia el **ciclo de vida** de los objetos en cada caso.

🕒 A) Composición **fuerte** (el ciclo de vida de los puntos va ligado a la línea)

Idea: Los puntos **solo existen como partes internas** de la **LíneaFuerte**. No se pueden crear ni mantener referencias a esos puntos desde fuera. Para modelarlo en Java:

- Hacemos una clase **LíneaFuerte** que **crea** sus “puntos extremos”.
- Esos puntos son una **clase interna no estática** **PuntoExtremo** con **constructor privado**.
- No exponemos las instancias internas; como mucho, devolvemos **copias por valor** (DTO) o solo las **coordenadas**.

Resultado: si la **LíneaFuerte** “desaparece” (se colecta), sus **PuntoExtremo** **también**. No pueden existir sin su línea.

```
// LineaFuerte.java
public final class LineaFuerte {
    private final PuntoExtremo inicio;
    private final PuntoExtremo fin;

    public LineaFuerte(double x1, double y1, double x2, double y2) {
        this.inicio = new PuntoExtremo(x1, y1);
        this.fin     = new PuntoExtremo(x2, y2);

        if (Double.compare(inicio.x, fin.x) == 0 && Double.compare(inicio.y,
fin.y) == 0) {
            throw new IllegalArgumentException("Una línea no puede tener extremos
idénticos");
        }
    }

    /** Longitud de la línea (distancia entre extremos) */
    public double longitud() {
        double dx = fin.x - inicio.x;
        double dy = fin.y - inicio.y;
        return Math.hypot(dx, dy);
    }

    // --- Accesores seguros (no exponen los objetos internos) ---
}
```

```

    public double getXInicio() { return inicio.x; }
    public double getYInicio() { return inicio.y; }
    public double getXFin()    { return fin.x; }
    public double getYFin()    { return fin.y; }

    /** Opcional: devolver una COPIA por valor (no el interno) */
    public Punto valorInicio() { return new Punto(inicio.x, inicio.y); }
    public Punto valorFin()    { return new Punto(fin.x, fin.y); }

    @Override
    public String toString() {
        return "LineaFuerte[" + inicio.x + ", " + inicio.y + ") -> (" + fin.x +
", " + fin.y + ")";
    }

    /**
     * Clase interna NO estática: está ligada a la instancia de LineaFuerte.
     * Constructor privado: NADIE fuera de LineaFuerte puede crearla.
     */
    private final class PuntoExtremo {
        private final double x;
        private final double y;

        private PuntoExtremo(double x, double y) {
            this.x = x;
            this.y = y;
        }
    }
}

```

Nota: He usado una clase `Punto` (inmutable y pública) solo como **valor de retorno** de copias, no para almacenar los extremos internos. Eso mantiene la **propiedad** y el **ciclo de vida** de los "puntos reales" dentro de `LineaFuerte`.

● B) Composición **débil** / **agregación** (los puntos existen independientemente)

Idea: `LineaDebil` se compone de dos `Punto` externos que pueden existir antes, ser compartidos por varias líneas y sobrevivir a que la línea deje de existir. La línea **no es dueña** de los puntos, solo los **referencia**.

Resultado: el ciclo de vida de `Punto` es **independiente** del de `LineaDebil`.

```

// Punto.java (valor inmutable reutilizable)
public final class Punto {
    private final double x;
    private final double y;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }
}

```

```

    }

    public double distanciaA(Punto otro) {
        if (otro == null) throw new IllegalArgumentException("El punto destino no
puede ser null");
        return Math.hypot(otro.x - this.x, otro.y - this.y);
    }

    public double getX() { return x; }
    public double getY() { return y; }

    @Override
    public String toString() { return "Punto(" + x + ", " + y + ")"; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Punto)) return false;
        Punto p = (Punto) o;
        return Double.compare(p.x, x) == 0 && Double.compare(p.y, y) == 0;
    }

    @Override
    public int hashCode() {
        return java.util.Objects.hash(x, y);
    }
}

```

```

// LineaDebil.java
public final class LineaDebil {
    private final Punto inicio;
    private final Punto fin;

    public LineaDebil(Punto inicio, Punto fin) {
        if (inicio == null || fin == null) {
            throw new IllegalArgumentException("Los puntos no pueden ser null");
        }
        if (inicio.equals(fin)) {
            throw new IllegalArgumentException("Una línea no puede tener extremos
idénticos");
        }
        this.inicio = inicio; // Referencia compartible
        this.fin = fin; // Referencia compartible
    }

    public double longitud() {
        return inicio.distanciaA(fin);
    }

    public Punto getInicio() { return inicio; } // Se devuelve la MISMA instancia
(es immutable)

```

```

    public Punto getFin()    { return fin; }

    @Override
    public String toString() {
        return "LineaDebil[" + inicio + " -> " + fin + "];"
    }
}

```

Pequeña demo comparando ambos enfoques

```

// Demo.java
public class Demo {
    public static void main(String[] args) {
        // --- Composición fuerte: la línea crea y posee sus extremos ---
        LineaFuerte lf = new LineaFuerte(0, 0, 3, 4);
        System.out.println(lf);
        System.out.println("Longitud (fuerte): " + lf.longitud());
        // Acceso por coordenadas o por copias
        System.out.println("Extremo inicio (copia): " + lf.valorInicio());

        // --- Composición débil: los puntos pueden existir y compartirse ---
        Punto p1 = new Punto(0, 0);
        Punto p2 = new Punto(3, 4);
        LineaDebil ld1 = new LineaDebil(p1, p2);
        LineaDebil ld2 = new LineaDebil(p1, new Punto(6, 8)); // p1 compartido
        System.out.println(ld1);
        System.out.println("Longitud (débil, ld1): " + ld1.longitud());
        System.out.println("Longitud (débil, ld2): " + ld2.longitud());
    }
}

```

Cómo compilar y ejecutar

```

javac Punto.java LineaDebil.java LineaFuerte.java Demo.java
java Demo

```

Diferencias clave (resumen conceptual)

- **Composición fuerte (◆ UML “composición”)**
 - La **LineaFuerte** **posee** sus extremos (**PuntoExtremo**).
 - Los “puntos” **no existen fuera** de la línea (clase interna no estática + constructor privado).
 - El ciclo de vida de las partes está **ligado** al del todo.
 - **No se comparten** entre líneas.

- **Composición débil** (◊ UML “agregación/asociación”)

- La **LíneaDébil** **referencia** **Punto** externos.
- Los puntos **pueden existir** sin la línea, **pueden compartirse**, y **sobreviven** a la línea.
- El ciclo de vida es **independiente**.

7. En Java, en la composición fuerte, ¿cuando el contenedor destruye los objetos? No se observa que **Línea** destruya los **Punto** explícitamente, ¿Por qué?

En **Java**, incluso en un diseño de **composición fuerte**, el contenedor **no “destruye” explícitamente** a los objetos parte.

La razón es que **Java no tiene destrucción determinista**: la **liberación de memoria** la realiza el **garbage collector (GC)** cuando **un objeto deja de ser alcanzable** (*unreachable*). Por eso, en tu ejemplo, no ves que **Línea** “destruya” los **Punto**.

¿Cuándo “desaparecen” los objetos en Java?

- Un objeto (por ejemplo, cada **Punto** interno de **Línea**) se vuelve **elegible para GC** cuando **ya no hay ninguna referencia** que permita alcanzarlo desde *raíces* del programa (hilos activos, pilas, variables estáticas, etc.).
- En **composición fuerte**, los puntos **solo viven** mientras la **Línea** que los contiene esté viva (haya referencias a ella).
 - Cuando la **Línea** **deja de ser alcanzable**, **sus puntos internos también** dejan de ser alcanzables ⇒ **GC los recogerá** en algún momento futuro.
- El momento exacto del GC **no es predecible** (no hay destructor ni **delete** como en C++).

Idea clave: la **composición fuerte** es una **propiedad de diseño** (propiedad/posesión y ciclo de vida ligado), **no un mecanismo de destrucción**. En Java, ese enlace de ciclo de vida se **modela** haciendo que las partes estén **encapsuladas** y **no se compartan**, de modo que **solo sobreviven** mientras el “todo” sea **alcanzable**.

¿Por qué **Línea** no “destruye” sus **Punto**?

- Porque **no existe** una operación de destrucción explícita en el lenguaje.
- No tiene sentido escribir algo como **punto = null**; “para destruir”: poner a **null** puede **ayudar** a perder la última referencia, pero **no destruye**; solo **adelanta** la **inaccesibilidad**. El **GC** es quien libera finalmente.

Matiz importante: memoria vs. recursos externos

El GC **solo** gestiona **memoria**. Si tus objetos manejan **recursos externos** (ficheros, sockets, conexiones...):

- **Debes** liberarlos **explícitamente** (p. ej., **close()**).
- Usa **try-with-resources** con tipos **AutoCloseable** para cierre determinista:

```
try (var in = Files.newInputStream(ruta)) {
    // usar el recurso
} // aquí se cierra sí o sí
```

- No confíes en `finalize()` (está **deprecado para eliminación**). En su lugar, usa cierres explícitos, *cleaners* si es imprescindible, o patrones RAII “a la Java” con `try-with-resources`.

Cómo modelar la composición fuerte en Java

Para que la “parte” **no pueda vivir** sin el “todo”:

- **Encapsula** las partes como **campos** `private final`.
- **No expongas** las instancias internas (si necesitas exponer datos, devuelve **copias** o **valores inmutables**).
- **Evita el compartir** la misma parte entre varios “todos”.
- Si usas clases **internas no estáticas** o constructores **privados** de las partes, refuerzas que **solo** el contenedor pueda crearlas.

Con estas pautas, cuando **nadie** conserve referencias a la **Línea**, **tampoco** a sus “puntos” ⇒ ambos quedan **inaccesibles** y el **GC** los liberará.

Resumen

- En Java **no** hay destrucción explícita: la **memoria** se libera por **GC** cuando los objetos son **inaccesibles**.
- En composición fuerte, las partes **mueren** cuando el “todo” deja de ser alcanzable.
- **Línea no destruye Punto** porque **no hay destructores**; el GC se encarga.
- Para **recursos externos**, usa **cierre explícito** y `try-with-resources`.

8. Pon un ejemplo de composicion débil entre un departamento que tiene varios profesores.

Implementa dos composiciones a la vez: entre el departamento y todos sus profesores y entre el departamento y su director, que es un profesor del departamento. Siempre debe haber un director en el departamento desde el inicio. Lanza excepciones si se viola la invariante. Emplea arrays primitivos de Java, estilo `Profesor[]`, con máximo 50, pero no rompas la encapsulación, no desveles que estás empleando un array, permite añadir un `Profesor` al final de la lista, y eliminar un profesor dada su posición. Da acceso a los profesores con un método para saber cuántos hay y otro para obtener un profesor por posición. El director se puede cambiar por otro profesor del departamento. Sin embargo, ten en cuenta esta invariante de clase: el director debe formar siempre parte de la lista de profesores, es decir, ten cuidado al cambiar el director o al eliminar un profesor.

Ejemplo completo en **Java** de **composición débil (agregación)** entre un **Departamento** que *tiene varios* **Profesor** y, además, *tiene un* **director** (que **siempre** debe ser uno de los profesores del departamento).

Se cumplen los requisitos:

- Internamente el departamento usa un **array primitivo** `Profesor[]` con **capacidad máxima 50**, pero **no se expone** al exterior (no rompemos la encapsulación).
- Se puede **añadir** un profesor **al final** de la lista.
- Se puede **eliminar** un profesor **por posición**.
- Métodos para **saber cuántos** profesores hay y **obtener un profesor por posición**.
- El **director siempre existe** desde el inicio y **siempre forma parte** de la lista de profesores.
- Cambiar el director **solo** puede hacerse por **otro profesor ya existente** en el departamento.
- Se lanzan **excepciones** si se viola la **invariante**.

Nota conceptual: Es **composición débil / agregación** porque `Departamento` *no es dueño* del ciclo de vida de `Profesor`: los `Profesor` pueden existir fuera del `Departamento` y podrían (en otro contexto) pertenecer también a otros.

Profesor.java (valor inmutable y con identidad)

```
public final class Profesor {
    private final String id;          // Identificador único (DNI, código interno,
    etc.)
    private final String nombre;

    public Profesor(String id, String nombre) {
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("El id del profesor no puede ser
nulo ni vacío");
        }
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre del profesor no puede
ser nulo ni vacío");
        }
        this.id = id;
        this.nombre = nombre;
    }

    public String getId() { return id; }

    public String getNombre() { return nombre; }

    @Override
    public String toString() {
        return "Profesor{id='" + id + "', nombre='" + nombre + "'}";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Profesor)) return false;
        Profesor that = (Profesor) o;
        // Igualdad basada SOLO en el id (identidad)
        return id.equals(that.id);
    }
}
```



```

@Override
public int hashCode() {
    return id.hashCode();
}
}

```

Departamento.java (agregación con invariante de director $\in$ profesores)

```

public final class Departamento {
    private static final int CAPACIDAD_MAX = 50;

    // --- Estado encapsulado ---
    private final Profesor[] profesores = new Profesor[CAPACIDAD_MAX];
    private int tam = 0; // número de elementos válidos en 'profesores'
    private Profesor director;

    /**
     * Crea un departamento con un director inicial.
     * Invariante: el director SIEMPRE está en la lista de profesores.
     */
    public Departamento(Profesor directorInicial) {
        if (directorInicial == null) {
            throw new IllegalArgumentException("El director inicial no puede ser
null");
        }
        // El director debe formar parte de la lista desde el inicio
        anadirProfesor(directorInicial);
        this.director = directorInicial;
        // Invariante cumplida: director  $\in$  profesores
    }

    // --- Consultas públicas (no rompen la encapsulación) ---

    /** Número actual de profesores. */
    public int numeroProfesores() {
        return tam;
    }

    /**
     * Devuelve el profesor en la posición indicada (0..numeroProfesores()-1).
     * No se expone la estructura interna; se entrega una referencia al objeto
(inmutable).
     */
    public Profesor profesorEn(int posicion) {
        comprobarRango(posicion);
        return profesores[posicion];
    }
}

```

```
/** Devuelve el director actual. */
public Profesor getDirector() {
    return director;
}

// --- Comandos públicos que respetan la invariante ---

/**
 * Añade un profesor al final de la lista.
 * Reglas:
 * - No null
 * - No duplicados (por id)
 * - Capacidad máxima 50
 */
public void anadirProfesor(Profesor profesor) {
    if (profesor == null) {
        throw new IllegalArgumentException("El profesor no puede ser null");
    }
    if (tam >= CAPACIDAD_MAX) {
        throw new IllegalStateException("Capacidad máxima alcanzada (" +
CAPACIDAD_MAX + ")");
    }
    if (contieneProfesor(profesor)) {
        throw new IllegalArgumentException("El profesor ya pertenece al
departamento: " + profesor.getId());
    }
    profesores[tam++] = profesor; // insertar al final
}

/**
 * Elimina el profesor en la posición indicada.
 * Reglas:
 * - No se puede eliminar al director (mantener invariante).
 * - Desplaza elementos para mantener el orden.
 * @return el profesor eliminado (por si se quiere registrar o informar).
 */
public Profesor eliminarProfesorEn(int posicion) {
    comprobarRango(posicion);
    Profesor eliminado = profesores[posicion];

    // Invariante: el director debe pertenecer SIEMPRE a la lista.
    if (eliminado.equals(director)) {
        // Sugerencia de uso: cambiar director antes de eliminar su posición
        throw new IllegalStateException(
            "No se puede eliminar al director actual. Cambie el director antes
de eliminarlo."
        );
    }

    // Desplazamiento hacia la izquierda
    for (int i = posicion; i < tam - 1; i++) {
        profesores[i] = profesores[i + 1];
    }
    profesores[--tam] = null; // ayudar al GC
}
```

```

        return eliminado;
    }

    /**
     * Cambia el director por otro profesor del departamento.
     * Reglas:
     * - El nuevo director debe estar en la lista de profesores.
     * - No null.
     * - Permite que sea el mismo (no haría nada).
     */
    public void cambiarDirector(Profesor nuevoDirector) {
        if (nuevoDirector == null) {
            throw new IllegalArgumentException("El nuevo director no puede ser
null");
        }
        if (!contieneProfesor(nuevoDirector)) {
            throw new IllegalArgumentException(
                "El nuevo director debe pertenecer al departamento: " +
nuevoDirector.getId()
            );
        }
        this.director = nuevoDirector;
        // Invariante preservada: director ∈ profesores
    }

    // --- Utilidades privadas ---

    private void comprobarRango(int posicion) {
        if (posicion < 0 || posicion >= tam) {
            throw new IndexOutOfBoundsException(
                "Posición fuera de rango: " + posicion + " (tamaño actual: " + tam
+ ")")
        };
    }

    private boolean contieneProfesor(Profesor p) {
        return indiceDe(p) != -1;
    }

    private int indiceDe(Profesor p) {
        for (int i = 0; i < tam; i++) {
            if (profesores[i].equals(p)) return i;
        }
        return -1;
    }
}

```

Demo.java (uso básico y verificación de la invariante)

```
public class Demo {
    public static void main(String[] args) {
        Profesor juan = new Profesor("P001", "Juan Pérez");
        Profesor ana = new Profesor("P002", "Ana Gómez");
        Profesor luca = new Profesor("P003", "Luca Ortega");

        // Siempre debe haber director desde el inicio
        Departamento dpt = new Departamento(juan);
        System.out.println("Director: " + dpt.getDirector());
        System.out.println("Profesores: " + dpt.numeroProfesores()); // 1

        // Añadir profesores al final
        dpt.anadirProfesor(ana);
        dpt.anadirProfesor(luca);
        System.out.println("Profesores: " + dpt.numeroProfesores()); // 3

        // Acceso por posición
        for (int i = 0; i < dpt.numeroProfesores(); i++) {
            System.out.println(i + ": " + dpt.profesorEn(i));
        }

        // Cambiar director a otro profesor del departamento
        dpt.cambiarDirector(ana);
        System.out.println("Nuevo director: " + dpt.getDirector());

        // Intentar eliminar al director -> excepción
        try {
            // Buscar la posición del director (aquí por demo, inspeccionamos)
            int posDirector = -1;
            for (int i = 0; i < dpt.numeroProfesores(); i++) {
                if (dpt.profesorEn(i).equals(dpt.getDirector())) {
                    posDirector = i; break;
                }
            }
            if (posDirector >= 0) {
                dpt.eliminarProfesorEn(posDirector);
            }
        } catch (IllegalStateException ex) {
            System.out.println("Esperado: " + ex.getMessage());
        }

        // Eliminar un profesor que NO es director
        // (p.ej., elimino la posición 2 si existe)
        if (dpt.numeroProfesores() > 2) {
            dpt.eliminarProfesorEn(2);
            System.out.println("Tras eliminar posición 2, total: " +
dpt.numeroProfesores());
        }

        // Para eliminar al director: primero cambiarlo a otro existente y luego
eliminar al anterior
        dpt.cambiarDirector(juan); // ahora Juan vuelve a ser director
        // (si quisiéramos eliminar a Ana ahora, ya no es directora)
```

```
}
}
```

Puntos clave del diseño

- **Composición débil (agregación):** `Departamento` mantiene **referencias** a `Profesor`, pero **no posee** su ciclo de vida. Un `Profesor` puede existir sin el `Departamento`.
- **Invariante de clase:**
 - `director != null`
 - `director` **está** en la lista de `profesores`.
- **Encapsulación:**
 - No se expone el `Profesor[]` interno.
 - Se ofrecen operaciones controladas:
 - `numeroProfesores()`
 - `profesorEn(int posicion)`
 - `anadirProfesor(Profesor)`
 - `eliminarProfesorEn(int posicion)`
 - `cambiarDirector(Profesor)`
- **Seguridad:** Sin **duplicados** por identidad (`id`), **capacidad** máxima, y **excepciones** claras ante violaciones de la invariante o mal uso.
- **Inmutabilidad de `Profesor`:** al ser inmutable, **devolver referencias** a profesores **no rompe** la encapsulación del estado.

9. En Java, existen también `List`, cambia y muestra cómo sería el código anterior empleando `List` en vez de arrays primitivos. ¿Qué parte del código original te has ahorrado? Además, fíjate en el método `getProfesor(int pos)`: si en su lugar existiera un método que devolviera todos los profesores a la vez, ¿qué problema tendría devolver directamente la lista interna? ¿Cómo lo resolverías?

¡Vamos a reescribir el ejercicio usando `List` en Java! Mantendremos las **mismas invariantes** y la **composición débil (agregación)**: el `Departamento` tiene una lista de `Profesor` y un **director** que **siempre** debe pertenecer a esa lista.

Además, te explico **qué código nos ahorramos** respecto a la versión con arrays, y cómo **exponer todos los profesores** sin romper la **encapsulación**.

1) Clases

`Profesor.java` (sin cambios)

Seguimos usando la clase inmutable con igualdad por `id`:

```
public final class Profesor {
    private final String id;           // Identificador único
```

```

    private final String nombre;

    public Profesor(String id, String nombre) {
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("El id del profesor no puede ser nulo ni vacío");
        }
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre del profesor no puede ser nulo ni vacío");
        }
        this.id = id;
        this.nombre = nombre;
    }

    public String getId() { return id; }
    public String getNombre() { return nombre; }

    @Override
    public String toString() {
        return "Profesor{id='" + id + "', nombre='" + nombre + "'}";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Profesor)) return false;
        Profesor that = (Profesor) o;
        return id.equals(that.id);
    }

    @Override
    public int hashCode() {
        return id.hashCode();
    }
}

```

Departamento.java (con List<Profesor>)

Implementamos las operaciones solicitadas con una List:

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public final class Departamento {
    private static final int CAPACIDAD_MAX = 50;

    private final List<Profesor> profesores; // Lista interna encapsulada
    private Profesor director;

```

```

/**
 * Crea un departamento con un director inicial.
 * Invariante: el director SIEMPRE está en la lista de profesores.
 */
public Departamento(Profesor directorInicial) {
    if (directorInicial == null) {
        throw new IllegalArgumentException("El director inicial no puede ser
null");
    }
    this.profesores = new ArrayList<>(CAPACIDAD_MAX);
    this.anadirProfesor(directorInicial); // añade y valida
duplicados/capacidad
    this.director = directorInicial;      // pertenece a la lista ->
invariante OK
}

/** Número actual de profesores. */
public int numeroProfesores() {
    return profesores.size();
}

/**
 * Devuelve el profesor en la posición indicada (0..numeroProfesores()-1).
 * Lanza IndexOutOfBoundsException si la posición no es válida (lo gestiona
List).
 */
public Profesor profesorEn(int posicion) {
    return profesores.get(posicion);
}

/** Devuelve el director actual. */
public Profesor getDirector() {
    return director;
}

/**
 * Añade un profesor al final de la lista.
 * Reglas:
 * - No null
 * - No duplicados (por equals/hashCode -> id)
 * - Capacidad máxima 50
 */
public void anadirProfesor(Profesor profesor) {
    if (profesor == null) {
        throw new IllegalArgumentException("El profesor no puede ser null");
    }
    if (profesores.size() >= CAPACIDAD_MAX) {
        throw new IllegalStateException("Capacidad máxima alcanzada (" +
CAPACIDAD_MAX + ")");
    }
    if (profesores.contains(profesor)) {
        throw new IllegalArgumentException("El profesor ya pertenece al
departamento: " + profesor.getId());
    }
}

```

```

    }
    profesores.add(profesor);
}

/**
 * Elimina el profesor en la posición indicada.
 * - No se puede eliminar al director (mantener invariante).
 * @return el profesor eliminado (opcionalmente útil para logs).
 */
public Profesor eliminarProfesorEn(int posicion) {
    Profesor eliminado = profesores.get(posicion); // deja que List valide
rango
    if (eliminado.equals(director)) {
        throw new IllegalStateException(
            "No se puede eliminar al director actual. Cambie el director antes
de eliminarlo."
        );
    }
    return profesores.remove(posicion);
}

/**
 * Cambia el director por otro profesor del departamento.
 * - Debe existir en la lista.
 * - No null.
 */
public void cambiarDirector(Profesor nuevoDirector) {
    if (nuevoDirector == null) {
        throw new IllegalArgumentException("El nuevo director no puede ser
null");
    }
    if (!profesores.contains(nuevoDirector)) {
        throw new IllegalArgumentException(
            "El nuevo director debe pertenecer al departamento: " +
nuevoDirector.getId()
        );
    }
    this.director = nuevoDirector;
    // Invariante: director ∈ profesores
}

// ----- Opciones para exponer "todos los profesores" sin romper
encapsulación -----

/**
 * Devuelve una vista inmodificable de los profesores.
 * Ventaja: no se puede alterar desde fuera.
 * Inconveniente: sigue exponiendo referencias a los objetos (que son
inmutables, así que OK).
 */
public List<Profesor> profesoresInmodificables() {
    return Collections.unmodifiableList(profesores);
}

```



```

/**
 * Devuelve una COPIA (defensiva) de la lista.
 * Nadie puede alterar la lista interna ni su orden.
 */
public List<Profesor> copiaProfesores() {
    return new ArrayList<>(profesores);
}
}

```

Demo.java

Uso básico, similar al anterior:

```

public class Demo {
    public static void main(String[] args) {
        Profesor juan = new Profesor("P001", "Juan Pérez");
        Profesor ana = new Profesor("P002", "Ana Gómez");
        Profesor luca = new Profesor("P003", "Luca Ortega");

        Departamento dpt = new Departamento(juan);
        System.out.println("Director: " + dpt.getDirector());
        System.out.println("Profesores: " + dpt.numeroProfesores()); // 1

        dpt.anadirProfesor(ana);
        dpt.anadirProfesor(luca);
        System.out.println("Profesores: " + dpt.numeroProfesores()); // 3

        for (int i = 0; i < dpt.numeroProfesores(); i++) {
            System.out.println(i + ": " + dpt.profesorEn(i));
        }

        dpt.cambiarDirector(ana);
        System.out.println("Nuevo director: " + dpt.getDirector());

        // Intento de eliminar al director -> excepción
        try {
            for (int i = 0; i < dpt.numeroProfesores(); i++) {
                if (dpt.profesorEn(i).equals(dpt.getDirector())) {
                    dpt.eliminarProfesorEn(i);
                }
            }
        } catch (IllegalStateException ex) {
            System.out.println("Esperado: " + ex.getMessage());
        }

        // Obtener lista inmodificable
        System.out.println("Profesores (vista inmodificable): " +
            dpt.profesoresInmodificables());

        // Eliminar a Luca si sigue estando en posición 2
    }
}

```

```

        if (dpt.numeroProfesores() > 2) {
            dpt.eliminarProfesorEn(2);
            System.out.println("Tras eliminar posición 2, total: " +
dpt.numeroProfesores());
        }
    }
}

```

2) ¿Qué parte del código original nos hemos ahorrado?

Al pasar de **array primitivo** a **List**:

1. Gestión manual de índices y desplazamientos

- Ya **no** necesitamos el bucle que movía elementos al eliminar (**for** desplazando a la izquierda).
- Tampoco hay que **anular** la última posición con **null** tras borrar.

2. Comprobaciones de rango

- Ya **no** necesitamos un método **comprobarRango**: **List.get(pos)** y **List.remove(pos)** lanzan **IndexOutOfBoundsException** automáticamente.

3. Búsquedas auxiliares

- Ya **no** hace falta **indiceDe** ni **contieneProfesor** manuales: usamos **list.contains(obj)** y **list.indexOf(obj)**.

4. Capacidad dinámica

- Con **ArrayList** **no** hay que crear un array fijo y controlar manualmente el tamaño (**tam**).
- Mantenemos, por requisito del ejercicio, un **límite lógico** de 50 (lo comprobamos antes de **add**), pero **no** tenemos que gestionar arrays ni su "compactación".

En resumen, el código queda **más corto y legible**: menos manejo de infraestructura y más foco en la **lógica de negocio** (la **invariante** del director y las **reglas** de adición/eliminación).

3) Sobre **getProfesor(int pos)** vs. "devolver todos los profesores"

Tu método **profesorEn(int pos)** **no rompe encapsulación** porque devuelve **un único elemento** (inmutable) y **no expone** la estructura interna.

Si, en cambio, tuvieras un método que **devuelve todos los profesores**:

✗ Problema de exponer la **lista interna** directamente

- Si devuelves la **misma List interna**, el código cliente podría:
 - **modificarla** (**add**, **remove**, **clear**), **rompiendo invariantes** (por ejemplo, dejar al director fuera).
 - **alterar el orden** o **duplicar** profesores sin pasar por tus validaciones.

Esto se llama **exposición de representación** (*representation exposure*): el estado interno queda vulnerable.

☑ Soluciones recomendadas

1. Devolver una vista inmodificable:

```
public List<Profesor> profesoresInmodificables() {  
    return Collections.unmodifiableList(profesores);  
}
```

- Ventaja: el cliente **no puede** modificar la lista.
- Si **Profesor** es inmutable (como aquí), **no hay riesgo** de que modifiquen los objetos.

2. Devolver una copia defensiva:

```
public List<Profesor> copiaProfesores() {  
    return new ArrayList<>(profesores);  
}
```

- Ventaja: incluso aunque el cliente modifique la copia, **tu estado interno no cambia**.
- Puedes combinarla con **List.copyOf(profesores)** (Java 10+) para devolver una **lista inmodificable** ya copiada:

```
public List<Profesor> snapshotProfesores() {  
    return List.copyOf(profesores); // inmodificable y copiada  
}
```

¿Cuál usar?

- Si quieres **mostrar** pero **prohibir cambios** → **unmodifiableList**.
- Si necesitas **aislarte por completo** de cualquier efecto externo → **copia defensiva** (idealmente inmodificable).

4) Resumen

- Con **List**, tu **Departamento** es **más simple** y limpio:
 - Sin bucles de desplazamiento, sin **tam**, sin comprobaciones de rango manual.
 - Usas **contains/indexOf/size/get/remove/add**.
- **Encapsulación** sólida:
 - No devuelvas la lista interna tal cual.
 - Usa **Collections.unmodifiableList** o **copia defensiva** (**new ArrayList<>(lista) / List.copyOf(lista)**).
- **Invariante** garantizada:
 - Siempre hay director.
 - El director siempre **pertenece** a la lista de profesores.
 - No se puede eliminar al director sin antes cambiarlo.

10. Al igual que ocurre con las excepciones en Java, que pueden encerrar causas (que son excepciones), de forma recursiva, suponen un tipo especial de composiciones, denominadas composiciones recursivas. Pon un ejemplo en Java de una **Persona**, que sea inmutable, y que tiene una madre, que es otra **Persona**. Haz un main con un ejemplo de uso con una familia de personas, desde el nieto hasta la abuela. Enumera algún otro ejemplo clásico de composiciones recursivas.

Ejemplo claro de **composición recursiva** en Java: una clase **inmutable Persona** que "tiene-una" madre, que a su vez es otra **Persona**. Igual que las excepciones pueden encadenar una **cause**, aquí una **Persona** puede referenciar a otra **Persona** (su madre), y así sucesivamente.

Nota sobre el modelo: Para que el "árbol" tenga un origen (y evitar cadenas infinitas), permitimos que la madre sea **opcional** (**null**), lo mismo que ocurre con **Throwable.getCause()**.

Código: **Persona** inmutable con composición recursiva

```
public final class Persona {
    private final String nombre;
    private final Persona madre; // puede ser null si se desconoce o estamos en el
                                // ancestro raíz

    /**
     * Crea una persona con nombre y madre opcional.
     * @param nombre nombre no nulo ni vacío
     * @param madre la madre (puede ser null)
     */
    public Persona(String nombre, Persona madre) {
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre no puede ser nulo ni
vacío");
        }
        this.nombre = nombre;
        this.madre = madre; // inmutable: nunca cambia
    }

    public String getNombre() { return nombre; }

    /** Puede devolver null si no se ha especificado madre. */
    public Persona getMadre() { return madre; }

    /**
     * Devuelve la línea de ascendencia materna en forma de texto, comenzando por
esta persona.
     * Ej: "Nieto -> Madre -> Abuela"
     */
    public String ascendenciaMaternaComoTexto() {
        StringBuilder sb = new StringBuilder();
        Persona actual = this;
```

```

        while (actual != null) {
            if (sb.length() > 0) sb.append(" -> ");
            sb.append(actual.nombre);
            actual = actual.madre;
        }
        return sb.toString();
    }

    /**
     * Profundidad de ascendencia materna (esta persona cuenta como 1).
     * Si solo está esta persona (sin madre), devuelve 1; si tiene madre, 2; etc.
     */
    public int profundidadMaterna() {
        int depth = 0;
        Persona actual = this;
        while (actual != null) {
            depth++;
            actual = actual.madre;
        }
        return depth;
    }

    @Override
    public String toString() {
        return "Persona{nombre='" + nombre + "'" + (madre != null ? ", madre='" +
        madre.nombre + "'" : "") + "}";
    }
}

```

Main con un ejemplo (nieto → madre → abuela)

```

public class Main {
    public static void main(String[] args) {
        // Ancestro raíz (abuela) sin madre conocida:
        Persona abuela = new Persona("Abuela Carmen", null);

        // La madre tiene como madre a la abuela:
        Persona madre = new Persona("María", abuela);

        // El nieto tiene como madre a María:
        Persona nieto = new Persona("Lucas", madre);

        // Imprimimos detalles
        System.out.println("Nieto: " + nieto);
        System.out.println("Madre del nieto: " + nieto.getMadre());
        System.out.println("Abuela del nieto: " + nieto.getMadre().getMadre());

        // Cadena de ascendencia materna
        System.out.println("Ascendencia (materna) del nieto: " +
        nieto.ascendenciaMaternaComoTexto());
    }
}

```

```

        // Profundidad (personas en la cadena: nieto, madre, abuela -> 3)
        System.out.println("Profundidad materna del nieto: " +
nieto.profundidadMaterna());
    }
}

```

Salida esperada (aprox.):

```

Nieto: Persona{nombre='Lucas', madre='María'}
Madre del nieto: Persona{nombre='María', madre='Abuela Carmen'}
Abuela del nieto: Persona{nombre='Abuela Carmen'}
Ascendencia (materna) del nieto: Lucas -> María -> Abuela Carmen
Profundidad materna del nieto: 3

```

Comentarios de diseño

- **Inmutabilidad:**
 - `Persona` es `final` y sus campos son `final` implícitamente (no hay setters).
 - Las referencias no cambian tras el constructor.
- **Composición recursiva:**
 - El campo `madre` es otra `Persona`. Esto hace que la estructura sea una **cadena** (o rama) recursiva.
- **Terminación:**
 - Permitimos `madre == null` como caso base (igual que `Throwable.getCause()` puede ser `null`).
- **Ciclos:**
 - En un modelo real se debería evitar crear **ciclos** (que alguien sea madre de sí misma indirectamente), lo cual puede requerir **validaciones** adicionales si las entidades se construyen a partir de datos externos.

Otros ejemplos clásicos de **composiciones recursivas**

1. **Excepciones encadenadas:** `Throwable` → `getCause()` (otra `Throwable`) → ...
2. **Árboles / AST:** un `Nodo` que tiene `List<Nodo>` como hijos (árbol sintáctico, árboles de decisión, etc.).
3. **Sistema de archivos (Composite):** `Directorio` que contiene `List<Entrada>` donde cada `Entrada` puede ser `Archivo` o `Directorio`.
4. **Expresiones aritméticas:** una `Expr` puede ser `Constante`, `Variable`, o `Operacion(Expr izq, Expr der)`.
5. **Menús y submenús:** `MenuItem` con `List<MenuItem>` para anidar niveles.
6. **DOM / XML / JSON:** nodos que contienen otros nodos/arrays/objetos de la misma jerarquía.
7. **Empleados–Jerarquía:** `Empleado` con referencia a su `manager` (otro `Empleado`), formando un árbol/jerarquía.

11. ¿Qué son las relaciones de composición "bidireccionales"? ¿Qué habría que hacer para implementar este tipo de relación en el ejemplo de **Profesor** y **Departamento**?

A grandes rasgos, una **composición (o agregación) bidireccional** es una relación "todo–parte" en la que **ambos extremos** mantienen una **referencia entre sí**:

- El **todo** (p. ej., **Departamento**) mantiene a sus **partes** (p. ej., una colección de **Profesor**).
- Cada **parte** (**Profesor**) mantiene una **referencia de vuelta al todo** (**Departamento**).

La clave está en **mantener la consistencia** de ambas direcciones: cuando añades/eliminas un profesor del departamento, **debes actualizar** también la referencia del **Profesor** a su **Departamento**, y viceversa. Esto exige encapsular bien las operaciones y evitar que se puedan "romper" las invariantes desde fuera.

A continuación te muestro **cómo implementar** el ejemplo **Profesor** ↔ **Departamento** con **bidireccionalidad**, respetando la **invariante**: *siempre debe haber director y el director debe pertenecer a la lista de profesores*.

Nota: Conceptualmente esto es una **agregación bidireccional** (composición débil), porque los **Profesor** pueden existir fuera del **Departamento**. Si quisieras **composición fuerte**, harías que solo el **Departamento** pudiese crear y poseer a sus **Profesor** (y su ciclo de vida quedaría ligado).

Diseño y reglas

- **Bidireccionalidad coherente:**
 - Al **añadir** un **Profesor** a un **Departamento**:
 - Lo agregamos a la lista del **Departamento**.
 - Ajustamos `profesor.departamento = this`.
 - Al **eliminar** un **Profesor** del **Departamento**:
 - Lo quitamos de la lista.
 - Ajustamos `profesor.departamento = null`.
- **Invariante del director:**
 - Siempre debe existir un **director** (no `null`) desde el constructor.
 - El **director** debe **pertenecer** a la lista de profesores.
 - No se puede **eliminar** al director; antes hay que **cambiar** de director.
- **Encapsulación:**
 - No exponer la lista interna: devolver **vista inmodificable** o **copia defensiva**.
 - El `setDepartamento` del **Profesor** no debe ser público; lo gestionará el **Departamento** al añadir/eliminar (para que **no desincronicen** la relación).
- **Sin duplicados** por identidad (`equals` por `id`).
- **Capacidad lógica** máxima (50) como en tu requisito original.

Código

Profesor.java (parte con back-reference al Departamento)

```
import java.util.Objects;
```

```

public final class Profesor {
    private final String id;
    private final String nombre;
    // back-reference (bidireccional): paquete-privado o privado con método de
    gestión controlado
    private Departamento departamento; // puede ser null si está "sin
    departamento"

    public Profesor(String id, String nombre) {
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("El id del profesor no puede ser
nulo ni vacío");
        }
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre del profesor no puede
ser nulo ni vacío");
        }
        this.id = id;
        this.nombre = nombre;
    }

    public String getId() { return id; }
    public String getNombre() { return nombre; }

    /**
     * Devuelve el departamento actual (puede ser null).
     * No hay setter público: el Departamento es quien gestiona la relación
    bidireccional.
     */
    public Departamento getDepartamento() {
        return departamento;
    }

    // ----- Gestión interna de la relación (package-private o private + inner
    friend) -----
    // No hacer pública: solo debe ser usada por Departamento para mantener
    consistencia.
    void __setDepartamentoInterno(Departamento nuevo) {
        this.departamento = nuevo;
    }

    @Override
    public String toString() {
        return "Profesor{id='" + id + "', nombre='" + nombre + "'"
            + (departamento != null ? ", depto='" + departamento.getNombre() +
            "'" : "")
            + "}";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Profesor)) return false;
        Profesor that = (Profesor) o;

```



```

        return id.equals(that.id);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}

```

Departamento.java (agregación bidireccional con invariantes)

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public final class Departamento {
    private static final int CAPACIDAD_MAX = 50;

    private final String nombre;
    private final List<Profesor> profesores;
    private Profesor director;

    public Departamento(String nombre, Profesor directorInicial) {
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre del departamento no puede ser nulo ni vacío");
        }
        if (directorInicial == null) {
            throw new IllegalArgumentException("El director inicial no puede ser null");
        }
        this.nombre = nombre;
        this.profesores = new ArrayList<>(CAPACIDAD_MAX);

        // Añadimos al director inicial usando el método controlado (mantiene bidirección)
        this.anadirProfesor(directorInicial);
        this.director = directorInicial;
        // Invariante: director ∈ profesores
    }

    public String getNombre() {
        return nombre;
    }

    public Profesor getDirector() {
        return director;
    }

    public int numeroProfesores() {
        return profesores.size();
    }
}

```

```

    }

    public Profesor profesorEn(int posicion) {
        return profesores.get(posicion);
    }

    /** Vista inmodificable para no exponer la representación interna. */
    public List<Profesor> profesoresInmodificables() {
        return Collections.unmodifiableList(profesores);
    }

    /** Copia defensiva si prefieres "snapshot" separado. */
    public List<Profesor> copiaProfesores() {
        return new ArrayList<>(profesores);
    }

    /**
     * Añade un profesor al departamento. Mantiene la relación bidireccional.
     * - No null
     * - No duplicados
     * - Capacidad máxima 50
     * - Si el profesor ya pertenece a otro departamento, opcionalmente puedes:
     *   (a) prohibirlo, o (b) moverlo. Aquí elegimos (a) por simplicidad.
     */
    public void anadirProfesor(Profesor profesor) {
        if (profesor == null) {
            throw new IllegalArgumentException("El profesor no puede ser null");
        }
        if (profesores.size() >= CAPACIDAD_MAX) {
            throw new IllegalStateException("Capacidad máxima alcanzada (" +
CAPACIDAD_MAX + ")");
        }
        if (profesores.contains(profesor)) {
            throw new IllegalArgumentException("El profesor ya pertenece a este
departamento: " + profesor.getId());
        }
        if (profesor.getDepartamento() != null && profesor.getDepartamento() !=
this) {
            throw new IllegalStateException(
                "El profesor ya pertenece a otro departamento: " +
profesor.getDepartamento().getNombre()
            );
        }

        profesores.add(profesor);
        profesor.__setDepartamentoInterno(this);
    }

    /**
     * Elimina al profesor en la posición indicada. Mantiene la bidireccionalidad.
     * - No se puede eliminar al director (invariante).
     * - Devuelve el profesor eliminado.
     */
    public Profesor eliminarProfesorEn(int posicion) {

```

```

        Profesor eliminado = profesores.get(posicion);

        if (eliminado.equals(director)) {
            throw new IllegalStateException(
                "No se puede eliminar al director actual. Cambie el director antes
de eliminarlo."
            );
        }

        profesores.remove(posicion);
        eliminado.__setDepartamentoInterno(null);
        return eliminado;
    }

    /**
     * Cambia el director por otro profesor del departamento.
     * - Debe pertenecer al departamento.
     * - No null.
     */
    public void cambiarDirector(Profesor nuevoDirector) {
        if (nuevoDirector == null) {
            throw new IllegalArgumentException("El nuevo director no puede ser
null");
        }
        if (!profesores.contains(nuevoDirector)) {
            throw new IllegalArgumentException(
                "El nuevo director debe pertenecer al departamento: " +
nuevoDirector.getId()
            );
        }
        this.director = nuevoDirector;
        // Invariante preservada: director ∈ profesores
    }

    @Override
    public String toString() {
        return "Departamento{" + nombre + ", director=" + director.getNombre()
            + ", nProfesores=" + profesores.size() + "}";
    }
}

```

Demo.java (uso y verificación rápida)

```

public class Demo {
    public static void main(String[] args) {
        Profesor juan = new Profesor("P001", "Juan Pérez");
        Profesor ana = new Profesor("P002", "Ana Gómez");
        Profesor luca = new Profesor("P003", "Luca Ortega");

        Departamento dpt = new Departamento("Informática", juan);
        System.out.println(dpt);
    }
}

```

```

        System.out.println("Director: " + dpt.getDirector());
        System.out.println("Departamento de Juan: " +
juan.getDepartamento().getNombre());

        dpt.anadirProfesor(ana);
        dpt.anadirProfesor(luca);
        System.out.println("Profesores totales: " + dpt.numeroProfesores());
        System.out.println("Depto de Ana: " + ana.getDepartamento().getNombre());

        // Cambio de director
        dpt.cambiarDirector(ana);
        System.out.println("Nuevo director: " + dpt.getDirector());

        // Intento de eliminar al director -> excepción
        try {
            for (int i = 0; i < dpt.numeroProfesores(); i++) {
                if (dpt.profesorEn(i).equals(dpt.getDirector())) {
                    dpt.eliminarProfesorEn(i);
                }
            }
        } catch (IllegalStateException ex) {
            System.out.println("Esperado: " + ex.getMessage());
        }

        // Eliminar a Luca (no es director)
        for (int i = 0; i < dpt.numeroProfesores(); i++) {
            if (dpt.profesorEn(i).equals(luca)) {
                dpt.eliminarProfesorEn(i);
                break;
            }
        }
        System.out.println("Profesores tras eliminar a Luca: " +
dpt.numeroProfesores());

        // Comprobación bidireccional: al eliminar, departamento del profesor
        queda a null
        System.out.println("Departamento de Luca tras eliminar: " +
luca.getDepartamento());
    }
}

```

Puntos clave al implementar relaciones bidireccionales

1. **Un único “punto de entrada”** para modificar la relación (aquí, los métodos de `Departamento`). Evita setters públicos que permitan a un lado cambiar el vínculo sin actualizar el otro.
2. **Métodos internos (friend-like)** para la referencia inversa (`__setDepartamentoInterno`). Mantén estos métodos **no públicos** (paquete-privado o `private` si usas clases anidadas).
3. **Consistencia atómica**: cada operación que cambia la relación **actualiza ambos lados** antes de dar la operación por válida.

4. **Evitar ciclos lógicos:** por ejemplo, que el mismo **Profesor** sea agregado dos veces; o que pertenezca a dos **Departamento** (decide si lo prohibes o si lo “mueves”).
 5. **No exponer colecciones internas:** devuelve **vistas inmodificables** o **copias**.
 6. **Invariantes claras y excepciones predictibles:** el contrato debe ser inequívoco (siempre hay director; el director pertenece a la lista; no se elimina al director).
-

Variantes y mejoras opcionales

- **“Mover” profesor entre departamentos:** podrías añadir `trasladarA(Profesor p, Departamento destino)` que:
 - verifica que `p` está en `this`,
 - si `p` es director, pide cambiar director antes,
 - hace `this.remove(p)` y `destino.add(p)`, manteniendo bidirección.
 - **IDs únicos en Departamento:** además de `contains`, podrías usar un `Set` auxiliar por `id` para detectar duplicados con complejidad $O(1)$.
 - **Hacer el back-reference inmutable en composición fuerte:** si fuese fuerte, **Profesor** se construiría solo desde **Departamento** y no podría vivir “huérfano”.
-
-